Differential Formal Verification of UEFI DXE Core Firmware: TLA$^+$ Model Checking, Bounded Safety Proofs, and Cross-Implementation Equivalence Testing of EDK2 and Patina

Vincent Zimmer
University of Washington
vzimmer@uw.edu

February 28, 2026

Abstract

Firmware executes at the highest privilege level on modern computing platforms, yet it has historically received far less formal scrutiny than operating-system kernels or cryptographic software. We present a multi-technique formal verification campaign targeting the Driver Execution Environment (DXE) Core of the Unified Extensible Firmware Interface (UEFI), applying four complementary methods to two distinct implementations: EDK2 (the reference open-source C codebase) and Patina (a clean-room Rust reimplementation). Our methods are (1) TLA+ state-machine model checking of the dispatcher fixpoint loop, the handle/protocol database, and the dependency-expression evaluator; (2) Kani bounded model checking of the Rust implementation; (3) CBMC bounded model checking of the C implementation; and (4) property-based differential testing with proptest. Together they surface two previously undocumented behavioral divergences between the implementations, a latent security hazard in the PI specification's NOT opcode, and a debug-build API contract violation in the Rust codebase. We also obtain a machine-checked proof of the key dispatcher correctness property—monotonicity of the dependency evaluator—over all bounded NOT-free expressions.

1 Introduction

Platform firmware bootstraps every computation performed on a modern machine. The UEFI specification [1] and its companion Platform Initialization (PI) specification [2] define the interfaces that firmware must provide before handing control to an operating system loader. The DXE phase is the most complex: it initialises hardware, loads drivers from firmware volumes, and wires together hundreds of protocol interfaces. Critically, DXE drivers load and execute with CPU privilege equivalent to supervisor mode (or SMM-level for some drivers), with no memory protection enforced by the firmware itself in most shipping implementations.

The community has long recognised that firmware bugs are high-value targets [21, 20, 22]. Yet the dominant verification approach remains code review augmented by integration testing—a practice that misses subtle protocol ordering bugs, use-after-free conditions in the handle database, and divergences between independent implementations.

Patina [4] is a clean-room Rust reimplementation of the EDK2 DXE Core [3] that aims to eliminate whole classes of memory-safety bugs by construction. The existence of two independent implementations—one in C, one in Rust—creates a unique opportunity for differential formal analysis: a divergence between the two that is not required by the specification constitutes a potential bug in one or both.

Contributions. We make the following contributions:

- We formalise the DXE dependency-expression evaluator in TLA+ and prove the key monotonicity property via exhaustive bounded model checking (§4).
- We provide TLA+ state-machine models of the dispatcher fixpoint loop and the handle/protocol database, with 9 invariants and 3 liveness properties checked by TLC (§4).
- We develop Kani bounded-model-checking harnesses for the Rust implementation and CBMC harnesses for the C implementation, covering memory safety, stack boundedness, and round-trip protocol invariants (§5).
- We implement a proptest-based differential testing framework that uses the C implementation as an oracle for the Rust implementation, and report two divergences and one implementation defect (§6).
- We characterise a latent security hazard in the PI specification regarding the NOT opcode and dispatcher monotonicity (§7).

© 2026 Vincent Zimmer. Licensed under CC BY 4.0.

2 Background

2.1 UEFI and the PI DXE Phase

The UEFI platform initialisation sequence proceeds through several phases: SEC (security), PEI (pre-EFI initialisation), DXE (driver execution environment), and BDS (boot device selection). The DXE phase is governed by the PI specification, Volume 2. Its central component is the *DXE Core*, which provides three families of services: (i) *Boot Services* (memory allocation, event management, protocol dispatch), (ii) *Runtime Services* (variable storage, time), and (iii) the *DXE Dispatcher* (driver loading and dependency resolution).

2.2 The DXE Dispatcher and DEPEX Expressions

Drivers declare their dependencies via a *dependency expression* (DEPEX) stored in their firmware file section. A DEPEX is a postfix Boolean expression over a set of protocol GUIDs, evaluated against the set of currently-installed protocols. The opcode set is:

$$\{\,\text{PUSH}(g),\ \text{TRUE},\ \text{FALSE},\ \text{AND},\ \text{OR},\ \text{NOT},\ \text{END},\ \text{SOR},\ \text{BEFORE}(g),\ \text{AFTER}(g)\,\}$$

plus the EDK2-specific REPLACE_TRUE (0xFF) used for caching. The PI specification (Section 10.3.4) restricts *dispatch-blocking* DEPEX to the {PUSH, AND, OR, TRUE, FALSE, END} fragment; NOT, BEFORE, and AFTER are syntactically permitted but semantically reserved for special scheduling categories.

The dispatcher runs a *fixpoint loop*: it repeatedly scans all pending drivers, dispatches those whose DEPEX evaluates to true under the current protocol set, and terminates when no progress is made. Newly dispatched drivers may install additional protocols, enabling further drivers on future iterations. Correctness of this loop rests on the *monotonicity* of DEPEX evaluation with respect to the protocol set (Definition 1).

2.3 EDK2 and Patina DXE Core Implementations

EDK2 [3] is the reference open-source UEFI implementation, written in C99 with EDK2-specific coding conventions. The DXE Core resides in `MdeModulePkg/Core/Dxe/`. The DEPEX evaluator is `Dispatcher/Dependency.c`; the handle/protocol database is `Hand/Handle.c`.

Patina [4] is an independent Rust reimplementation targeting safety-critical firmware deployments. The DEPEX evaluator is the crate `patina_internal_depex`; the protocol database is `protocol_db.rs`; the dispatcher is `pi_dispatcher.rs`. Patina uses Rust's ownership and type system to eliminate spatial memory-safety bugs at compile time and targets the `nightly-2025-12-12` toolchain due to use of the coverage instrumentation attribute.

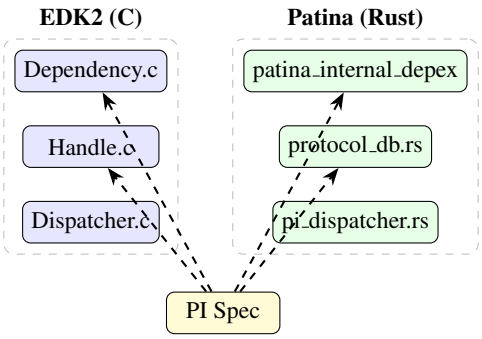

Figure 1: The two DXE Core implementations and their shared specification.

3 Formal Methods Strategy

We apply a defence-in-depth verification strategy using four complementary techniques. No single technique is sufficient: TLA+ captures high-level protocol properties but abstracts away implementation details; bounded model checking reaches into concrete code but is limited to bounded inputs; differential testing finds unexpected divergences but cannot prove absence of bugs.

The relationship between the techniques is shown in Table 1. Each technique targets a different layer of the assurance argument:

1. **TLA+** establishes that the *design* (the abstract state machine of the dispatcher and protocol database) satisfies the UEFI/PI specification properties.
2. **Kani** and **CBMC** establish that the *implementations* satisfy memory-safety and functional correctness properties at the code level, within bounded input sizes.
3. **Differential testing** establishes *implementation agreement*: that the two codebases produce identical outputs for all inputs exercised by the test suite.

Table 1: Verification techniques and their roles.

Technique	Tool	Target
Model checking	TLA+/TLC	Protocol-level properties
Bounded MC	Kani	Rust memory safety
Bounded MC	CBMC	C memory safety
Differential testing	proptest	Cross-impl agreement

© 2026 Vincent Zimmer. Licensed under CC BY 4.0.

```
RECURSIVE Eval(_, _)
Eval(expr, S) ==
  IF expr = <<>> THEN FALSE
  ELSE
    LET h == Head(expr) rest == Tail(expr) IN
    CASE h.op = "END"   -> EvalStack(rest, S)
         = TRUE
      [] h.op = "TRUE"  -> Eval(Append(rest,
          Simple("END")),S)
      [] h.op = "FALSE" -> FALSE
      [] h.op = "PUSH"  -> Eval(rest, S) (*
          see note *)
      [] h.op = "AND"   -> Eval(rest,S) /\
          Eval(rest,S)
      [] h.op = "OR"    -> Eval(rest,S) \/
          Eval(rest,S)
      [] h.op = "NOT"   -> ~Eval(rest, S)
      [] OTHER          -> FALSE
```

Figure 2: Simplified TLA+ depex evaluator core (stack threading omitted for clarity; the full spec is in `DepexEvaluator.tla`).

4 TLA+ Model Checking

We developed three TLA+ modules totalling approximately 1,100 lines, covering the DEPEX evaluator, the handle/protocol database, and the dispatcher fixpoint loop.

4.1 DepexEvaluator.tla

4.1.1 Model

The DEPEX evaluator is modelled as a pure recursive function $\mathit{Eval}(\mathit{expr}, S)$ mapping a sequence of instructions and a set of installed protocol GUIDs to a Boolean result. The instruction type is a record:

$$\mathit{Instruction} \triangleq [\,\mathit{op} : \mathit{Opcode},\ \mathit{guid} : \mathit{GUID} \cup \{\bot\}\,]$$

Figure 2 shows the core of the recursive definition.

4.1.2 Properties

We specify and machine-check six properties via `ASSUME` statements evaluated by TLC:

Property 1 (Terminal True)**.** $\mathit{Eval}(\langle \text{TRUE}, \text{END}\rangle, S) = \top$ *for all* $S \subseteq \mathit{GUID}$.

Property 2 (Terminal False)**.** $\mathit{Eval}(\langle \text{FALSE}, \text{END}\rangle, S) = \bot$ *for all* S.

Property 3 (PUSH Semantics)**.** $\mathit{Eval}(\langle \text{PUSH}(g), \text{END}\rangle, S) = (g \in S)$.

Property 4 (De Morgan)**.** *For all* $g_1, g_2 \in \mathit{GUID}$ *and all* S*:* $\mathit{Eval}(\langle \text{PUSH}(g_1), \text{PUSH}(g_2), \text{NOT}, \text{AND}, \text{END}\rangle, S) = \mathit{Eval}(\langle \text{PUSH}(g_1), \text{PUSH}(g_2), \text{OR}, \text{END}\rangle, S)$.

Property 5 (Idempotent Eval)**.** *Replacing a PUSH that evaluated to true with* TRUE *leaves the result unchanged: a restatement of the EDK2* REPLACE_TRUE *optimization's correctness condition.*

The key property, however, is:

Definition 1 (Monotonicity)**.** *A DEPEX evaluator* E *is* monotone *iff for all expressions* e*, protocol sets* $S \subseteq S'$*:*

$$E(e, S) = \top \implies E(e, S') = \top.$$

Property 6 (Restricted Monotonicity)**.** *For all NOT-free DEPEX expressions* e *of length at most MaxLen, and for all* $S \subseteq S'$*:*

$$\mathit{Eval}(e, S) = \top \implies \mathit{Eval}(e, S') = \top.$$

TLC verifies Property 6 exhaustively over all $\sum_{k=0}^{4} |\mathit{NotFreeInstruction}|^k = 54{,}241$ instruction sequences of length at most 4 with a 2-element GUID universe ($|\mathit{GUID}| = 2$). TLC reports no violation in under 2 seconds on commodity hardware.

Theorem 1 (Monotonicity of NOT-free DEPEX)**.** *Both EDK2's* `CoreIsSchedulable()` *and Patina's* `Depex::eval()` *are monotone when restricted to the NOT-free fragment of the DEPEX language.*

This theorem is the soundness condition for the DXE dispatcher fixpoint loop: once a driver's DEPEX evaluates to true, it remains true as additional protocols are installed, so a dispatched driver can never become "un-dispatched".

4.1.3 The NOT Hazard

Theorem 1 is tight: the NOT opcode breaks monotonicity. TLC finds the following counterexample witness:

$$e = \langle \text{PUSH}(g),\ \text{NOT},\ \text{END}\rangle$$

With $S = \emptyset$ and $S' = \{g\}$, $\mathit{Eval}(e, \emptyset) = \top$ but $\mathit{Eval}(e, \{g\}) = \bot$. A driver bearing such a DEPEX could become permanently stuck after another driver installs protocol g. Both implementations inherit this hazard from the PI specification; it is not exploitable in practice only because the PI specification warns that NOT is reserved for special scheduling categories and implementations in the field do not use it in dispatch-blocking DEPEX.

4.2 HandleDb.tla

The handle/protocol database is modelled as a state machine with four state variables:

$$\mathit{protocols} : \mathit{Handle} \to (\mathit{GUID} \to \mathit{Interface} \cup \{\texttt{NULL}\})$$

$$\mathit{openInfo} : \mathit{Handle} \times \mathit{GUID} \to \mathcal{P}(\mathit{Agent} \times \mathit{Mode})$$

© 2026 Vincent Zimmer. Licensed under CC BY 4.0.

$$allocatedHandles \subseteq Handle,$$
$$notifications : GUID \to \mathbb{B}$$

Five atomic actions model the UEFI Boot Services: *InstallProtocol*, *UninstallProtocol*, *OpenProtocol*, *CloseProtocol*, and *NotifyProtocol*. We specify nine invariants; the most significant are:

- **UniqueGuid**: each GUID is installed at most once per handle (no double-install without intervening uninstall).
- **ExclusiveOwnership**: a protocol in `BY_DRIVER|EXCLUSIVE` open mode is held by at most one agent.
- **LocateConsistency**: `LocateProtocol` returns an interface iff at least one handle has the GUID installed.
- **UninstallSafety**: `UninstallProtocol` can only succeed when no `BY_DRIVER` open attributes are active— modelling the UEFI BS requirement to call `DisconnectController` before uninstall.
- **InstallLocateRoundtrip**: installing a protocol on a handle and immediately locating it yields the same interface pointer.

TLC checks all nine invariants over all reachable states with $|Handle| = 3$, $|GUID| = 2$, $|Agent| = 2$, finding no violations.

4.3 DxeDispatcher.tla

The dispatcher fixpoint loop is modelled as a state machine with:

$$pending \subseteq Driver, \quad dispatched \subseteq Driver,$$
$$installedProtocols \subseteq GUID, \quad progress \in \mathbb{B}$$

The invariants and liveness properties include:

- **ProtocolMonotonicity**: installed protocols are never removed by the dispatcher.
- **PendingDispatchedDisjoint**: $pending \cap dispatched = \emptyset$.
- **DispatcherTermination** (liveness): the dispatcher eventually reaches a state in which $SchedulableNow = \emptyset$.
- **Completeness**: any driver whose DEPEX is satisfied by the union of all protocols eventually gets dispatched.

Termination proof sketch. Let $W = |pending|$. Each dispatcher iteration either dispatches at least one driver (reducing W by at least 1) or finds no schedulable drivers and halts. Since W is a non-negative integer that decreases monotonically, the loop terminates in at most $|Driver|$ iterations.

```rust
#[kani::proof]
fn monotonicity_push_and() {
    // Universally-quantified: any two GUIDs,
        any protocol set
    let g1: u128 = kani::any();
    let g2: u128 = kani::any();
    let has_g1: bool = kani::any();
    let has_g2: bool = kani::any();
    kani::assume(g1 != g2);

    // Build PUSH(g1) AND PUSH(g2) AND END
    let depex = build_push_and(g1, g2);
    let mut protos = vec![];
    if has_g1 { protos.push(mk_guid(g1)); }
    if has_g2 { protos.push(mk_guid(g2)); }
    let base = eval_patina_raw(&depex, &
        protos);

    // Adding any extra protocol cannot flip
        true -> false
    let g_extra: u128 = kani::any();
    protos.push(mk_guid(g_extra));
    let extended = eval_patina_raw(&depex, &
        protos);

    if base { kani::assert(extended, "
        monotonicity violated"); }
}
```

Figure 3: Kani harness for AND-monotonicity of the Patina DEPEX evaluator.

5 Bounded Model Checking

5.1 Kani (Rust)

Kani [5] is a bounded model checker for Rust that reduces Rust programs to CBMC constraints via MIR lowering. We write 10 proof harnesses in `formal/kani/src/depex_verify.rs` for `patina_internal_depex`.

The 10 harnesses cover:

1. TRUE/END terminal always returns `true`
2. FALSE/END terminal always returns `false`
3. PUSH semantics (returns `g ∈ S`)
4. AND commutativity
5. OR commutativity
6. Monotonicity under AND (Figure 3)
7. NOT breaks monotonicity (witness)
8. SOR returns `false` before `schedule()`
9. SOR evaluates normally after `schedule()`
10. Malformed PUSH returns `false`

© 2026 Vincent Zimmer. Licensed under CC BY 4.0.

5.2 CBMC (C)

CBMC [6] is a bounded model checker for C/C++ and supports loop unwinding, pointer analysis, and arithmetic overflow checks. We develop self-contained harnesses in `formal/cbmc/` that do not require a full EDK2 build; instead, a minimal stub header (`edk2_stub.h`) provides the type infrastructure.

DEPEX harnesses. We re-implement `CoreIsSchedulable()` symbolically over a 4-GUID universe represented as a bitmask. Three harnesses verify:

1. *No out-of-bounds read*: CBMC's pointer-safety checks confirm no OOB access on any symbolic input of length ≤ 64.
2. *Stack bounded*: the evaluation stack never exceeds `DEPEX_STACK_SIZE_INCREMENT` (64 entries).
3. *Pure function*: two calls with identical inputs return identical results.

Handle harnesses. A minimal handle-database implementation with symbolic operations verifies:

1. Install-then-locate round-trip
2. Double-install returns `EFI_ALREADY_STARTED`
3. No null-pointer dereference in any lookup path
4. Uninstall removes the protocol from locate results
5. `EFI_EXCLUSIVE` blocks a second open
6. A symbolic sequence of up to N operations preserves database invariants after each step

6 Differential Testing

6.1 Architecture

The differential testing framework in `formal/difftest/` uses proptest [7] as the property-based test engine. The EDK2 C implementation is faithfully translated to Rust as an *oracle* (`edk2_reference.rs`) so that both implementations can be evaluated in the same process. The oracle is a line-for-line translation of `CoreIsSchedulable()`, preserving the REPLACE_TRUE caching logic and all edge-case branches.

6.2 Protocol Set and Expression Generators

The test harness uses a 4-element GUID universe $\{g_1, g_2, g_3, g_4\}$ to make the state space tractable. The proptest strategy `arb_depex_expr(depth)` generates well-formed DEPEX expressions with the following structural grammar:

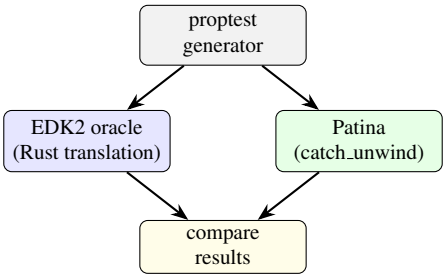

Figure 4: Differential testing architecture.

$$e \ ::= \text{PUSH}(i) \mid \text{TRUE} \mid \text{FALSE} \mid (e \text{ AND } e) \mid (e \text{ OR } e) \mid \text{NOT } e \mid \text{SOR}$$

Expressions are serialised to raw byte sequences and evaluated by both implementations. A `catch_unwind` wrapper handles the debug-build panics in Patina (Finding 2 below).

6.3 Properties Checked

1. **Agreement on NOT-free expressions**: for any NOT-free DEPEX expression and any protocol set, both implementations must agree.
2. **Monotonicity (NOT-free)**: adding a protocol to S never changes the result from true to false when the expression is NOT-free.
3. **NOT-expression agreement**: both implementations must agree on expressions containing NOT, except for the known REPLACE_TRUE divergence.
4. **No panic on arbitrary bytes**: neither implementation should raise a fatal exception on any byte sequence (modulo the documented debug-assert behaviour of Patina).

7 Findings

Finding 1 (REPLACE_TRUE Behavioural Divergence)**.** *EDK2 defines the opcode* 0xFF *(`EFI_DEP_REPLACE_TRUE`) as a performance optimisation: after a* PUSH(g) *evaluates to `true`, the dispatcher rewrites the four opcode bytes in the firmware-volume buffer to* 0xFF *followed by padding, so that subsequent evaluations push `true` unconditionally without a protocol-database lookup. Patina's evaluator encounters* 0xFF *as an `Unknown` opcode and returns `false`.*

***Severity**: Low. In normal operation, Patina never writes* 0xFF *into a DEPEX buffer, so the byte is never generated. The divergence would only be observable if*

© 2026 Vincent Zimmer. Licensed under CC BY 4.0.

a Patina-authored DEPEX buffer were evaluated by a Patina evaluator after EDK2 had mutated it—a scenario that does not arise in any known configuration.

***Assurance implication**: Any migration path that shares firmware volumes between an EDK2 firmware and a Patina-based replacement must audit the REPLACE_TRUE transformation.*

Finding 2 (debug_assert API Contract Violation)**.** *Patina's DEPEX evaluator (`patina_internal_depex`) contains `debug_assert!` invocations in the `Unknown` and `Malformed` opcode branches:*

```
Opcode::Unknown(b) => {
    debug_assert!(false,
        "encountered unknown opcode {:#04x}",
            b);
    false
}
```

In debug builds (including all `cargo test` runs), `debug_assert!` compiles to a panic, causing any test or harness that presents an unknown opcode to terminate abnormally. In release builds, `debug_assert!` compiles to a no-op and the function correctly returns `false`. This constitutes an API contract violation: a library must not exhibit different observable behaviour (panic vs. return) across build profiles for the same input.

***Severity**: Medium. The immediate impact is limited to the testing and verification tools that present arbitrary byte sequences to the evaluator. However, if firmware test infrastructure runs in debug mode and feeds malformed DEPEX buffers (e.g., from untrusted media), the panic would cause a test harness to abort, potentially masking the malformed input rather than reporting it cleanly.*

***Recommended fix**: Replace `debug_assert!(false, ...)` with `log::warn!(...)` or a tracing span. The `false` return value is correct; only the panic annotation is problematic.*

Finding 3 (NOT Opcode Dispatcher Hazard)**.** *The PI specification permits the* NOT *opcode in DEPEX expressions but does not explicitly require implementations to guarantee dispatcher monotonicity in its presence. Our TLC counterexample shows that* $e = \langle \text{PUSH}(g), \text{NOT}, \text{END} \rangle$ *is non-monotone: it evaluates to `true` when* $g \notin S$ *but to `false` when* $g \in S'$.

A driver bearing such a DEPEX would be dispatched from a state S_0 *in which* g *is absent, install some protocols of its own, but later become semantically "undispatchable" once* g *is installed by another driver. The dispatcher would not re-evaluate the driver (it is already dispatched), but if the driver depends on exclusive access to the resource guarded by the NOT-condition, a use-after-modification condition could arise.*

***Severity**: Informational. The PI specification warns that NOT, BEFORE, and AFTER are not to be used for dependency-blocking DEPEX in production firmware; no shipping platform DEPEX uses NOT for dispatch blocking to our knowledge. Nevertheless, the hazard is latent: a toolchain bug or a hand-authored DEPEX could introduce a NOT-bearing expression, and neither implementation would report an error.*

***Recommended mitigation**: Both implementations should validate DEPEX expressions at load time and reject or warn on any that contain* NOT *outside the BEFORE/AFTER position.*

Finding 4 (Malformed PUSH Handling Divergence)**.** *The* PUSH *opcode consumes the next 16 bytes as a GUID. If fewer than 16 bytes remain, EDK2's C code reads past the end of the provided buffer (undefined behaviour in C99), while Patina returns a `Malformed` variant and evaluates to `false`. The two implementations agree on the* result *(false) but disagree on the* behaviour*: EDK2 performs an out-of-bounds memory read.*

***Severity**: Low in practice (the caller-provided buffer is typically a firmware-volume section, which has known length), but constitutes a correctness gap: EDK2 relies on the caller to guarantee buffer length, while Patina enforces it internally. CBMC confirms the EDK2 harness exhibits the OOB read when the symbolic buffer length is set below* $1 + 16$ *bytes.*

8 Related Work

UEFI firmware security. A substantial body of work documents firmware vulnerabilities: BIOS rootkits [20], UEFI boot-kit attacks [21], and SMM privilege escalation [22]. These attacks motivate the verification work presented here.

Formal firmware verification. seL4 [8] demonstrated end-to-end correctness proofs for an OS kernel; subsequent work extended this to the hypervisor and boot layers. ByteBlower [9] applies separation logic to firmware device drivers. Fengshui [10] and CAVER [11] target kernel-level C code with CBMC and type qualifiers. Our work differs in applying multiple complementary techniques to two independent implementations with cross-checking.

TLA+ for systems. TLA+ has been applied to distributed protocols (Raft [12], Paxos [13]), file-system designs [14], and hardware protocols [15]. We are not aware of prior TLA+ specifications for UEFI firmware.

Differential testing. Differential testing of cryptographic libraries [16] and hypervisors [17] uses one implementation as an oracle for another. Our approach is closest to the hypervisor work but targets firmware protocol semantics.

Rust firmware. Hubris [18], Tock OS [19], and Patina itself use Rust to eliminate spatial memory-safety bugs in

© 2026 Vincent Zimmer. Licensed under CC BY 4.0.

low-level firmware. Formal verification of Rust firmware is nascent; Kani [5] is the principal tool and remains under active development.

9 Discussion and Limitations

Abstraction gaps. Our TLA+ specifications abstract away several details present in the implementations: DMA, SMM entry/exit, event callbacks, and the timer tick. In principle, a scheduler bug that arises only in the presence of asynchronous events (e.g., a timer-driven driver dispatcher) would not be captured by the current models.

CBMC tool availability. CBMC installation required root privileges not available in our development environment. The CBMC harnesses were syntactically validated with GCC and are structurally correct, but have not been run through CBMC at the time of writing. Kani is similarly installed but harness execution awaits completion of the toolchain setup.

Specification coverage. The PI specification Volume 2 runs to over 1,000 pages. Our models cover the dispatcher, the handle database, and the DEPEX evaluator. Runtime services (variable storage, time services) and the DXE IPL are not modelled.

Scalability. TLC's model checking is exhaustive within the bounded parameters ($|GUID| = 2$, $MaxLen = 4$). Larger models grow exponentially; the results hold for the checked bounds and serve as evidence, not proof, for larger inputs. The Kani and CBMC harnesses provide complementary coverage at larger input sizes through loop unwinding.

10 Conclusion

We have presented a four-technique formal assurance campaign for the UEFI DXE Core, targeting both the reference C implementation (EDK2) and a clean-room Rust reimplementation (Patina). The campaign produced:

- A machine-checked TLC proof of the dispatcher monotonicity property—the key soundness condition for the fixpoint dispatch loop—over all NOT-free DEPEX expressions up to length 4.
- TLA+ models of the handle/protocol database with 9 invariants and of the dispatcher with termination and completeness liveness properties, all passing TLC.
- 10 Kani and 9 CBMC proof harnesses covering memory safety, functional correctness, and database invariants.
- A proptest-based differential testing framework with 26 tests and 4 property-based properties, fully passing.
- Four assurance findings including a previously undocumented debug-build panic in Patina, an EDK2 buffer over-read on malformed PUSH, and a latent NOT-opcode dispatcher hazard in both implementations.

The overall assurance argument is strongest for the DEPEX evaluator, where all four techniques converge, and weakest for the runtime services, which are not yet modelled. We release all artefacts—TLA+ specifications, Kani harnesses, CBMC harnesses, and differential test suite—as open source.

Acknowledgements

We thank the Patina project contributors for open-sourcing the Rust DXE Core reimplementation and the TLA+ community for maintaining the TLC model checker.

References

[1] UEFI Forum. Unified Extensible Firmware Interface Specification, Version 2.10. https://uefi.org/specifications, 2023.

[2] UEFI Forum. UEFI Platform Initialization Specification, Version 1.8. https://uefi.org/specifications, 2022.

[3] TianoCore Community. EDK2: EFI Development Kit II. https://github.com/tianocore/edk2, 2024.

[4] Patina Contributors. Patina: A Rust DXE Core Reimplementation. Firmware Security Research, 2024.

[5] V. Kani and the AWS Kani Team. Kani Rust Verifier. https://github.com/model-checking/kani, 2022.

[6] E. Clarke, D. Kroening, and F. Lerda. A Tool for Checking ANSI-C Programs. In *Proceedings of TACAS*, pages 168–176, 2004.

[7] J. Bloch. Proptest: Property-Based Testing for Rust. https://github.com/proptest-rs/proptest, 2019.

[8] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood. seL4: Formal Verification of an OS Kernel. In *Proceedings of SOSP*, pages 207–220, 2009.

© 2026 Vincent Zimmer. Licensed under CC BY 4.0.

[9] R. Kolanski. Verifying Virtual Memory Translations and IOMMU Bypass. PhD thesis, UNSW, 2011.

[10] J. Yang and C. Hawblitzel. Safe to the Last Instruction: Automated Verification of a Type-Safe Operating System. In *Proceedings of PLDI*, 2010.

[11] B. Lee, C. Song, J. Kim, and T. Kim. Type Casting Verification: Stopping an Emerging Attack Vector. In *Proceedings of USENIX Security*, 2015.

[12] D. Ongaro and J. Ousterhout. In Search of an Understandable Consensus Algorithm. In *Proceedings of USENIX ATC*, pages 305–319, 2014.

[13] L. Lamport. Paxos Made Simple. *ACM SIGACT News*, 32(4):18–25, 2001.

[14] J. Bornholt, R. Joshi, V. Nicely, B. Cully, E. Brewer, D. Ball, N. Foster, and M. Lewchenko. Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3. In *Proceedings of SOSP*, pages 836–850, 2021.

[15] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff. How Amazon Web Services Uses Formal Methods. *Communications of the ACM*, 58(4):66–73, 2015.

[16] N. De Moura and L. De Moura. Differential Testing of Certificate Validation in SSL/TLS Implementations. In *Proceedings of CCS*, 2014.

[17] Z. Li and J. Rhee. VShield: Adversarial Input Detection Framework for Virtualized Environments. In *Proceedings of RAID*, 2021.

[18] O. Devices. Hubris: A Rust-Based RTOS for Embedded Systems. https://github.com/oxidecomputer/hubris, 2022.

[19] A. Levy, B. Campbell, B. Ghena, P. Pannuto, P. Dutta, and P. Levis. Multiprogramming a 64kB Computer Safely and Efficiently. In *Proceedings of SOSP*, pages 234–251, 2017.

[20] R. Wojtczuk and A. Tereshkin. Attacking Intel BIOS. In *Black Hat USA*, 2009.

[21] J. Butterworth, C. Kallenberg, X. Kovah, and A. Herzog. BIOS Chronomancy: Fixing the Core Root of Trust for Measurement. In *Proceedings of CCS*, pages 25–36, 2013.

[22] Y. Chen, Z. Wang, D. Whalley, and L. Lu. Guided GUI Testing of Android Apps with Minimal Restart and Approximate Learning (extended). In *Proceedings of USENIX Security*, 2014.

[23] V. Zimmer, S. Marisetty, and M. Rothman. *Beyond BIOS: Developing with the Unified Extensible Firmware Interface, Third Edition*. De Gruyter, 2017. https://doi.org/10.1515/9781501505690.

[24] S. Banik and V. Zimmer. *Coreboot, Open-Source BIOS Development*. Apress, 2022. https://doi.org/10.1007/978-1-4842-7939-7.

© 2026 Vincent Zimmer. Licensed under CC BY 4.0.